

Relatório do EP4 de MAC0425 - Inteligência Artificial

Análise comparativa de desempenho dos algoritmos implementados

Laufernando Souza Dias - 11222947

18 de julho de 2026

1 Metodologia

Os experimentos foram executados por meio de um script em bash que automatiza as chamadas a `gridworld.py` e `pacman.py` com os parâmetros especificados no enunciado, salvando a saída de cada execução em um arquivo de texto individual. Cada comando foi encapsulado sob um mecanismo de *timeout*, necessário porque um dos cenários testados (MDP 5, Seção 2.1) não satisfaz as condições teóricas de convergência da Iteração de Valor e gera episódios de duração ilimitada durante a simulação. Três blocos de experimentos foram realizados:

1. **Iteração de Valor no DiscountGrid:** 100 iterações (`-i 100`) seguidas da simulação de 10 episódios (`-k 10`) sob a política gulosa derivada de V , para cada uma das cinco combinações (*living reward*) especificadas na Tabela 1 do enunciado.
2. **Iteração de Valor vs. Q-Learning no CliffGrid:** 100 episódios de cada algoritmo, com os parâmetros padrão do simulador (sem alterar γ , ruído ou recompensas).
3. **Q-Learning no Pac-Man:** agente `PacmanQAgent` treinado por 2000 episódios (`-x 2000`), seguido da exibição de 10 episódios de teste com a política aprendida (*greedy*, $\epsilon = \alpha = 0$), nos mapas `smallGrid` e `tinyCorners`, testando $\alpha \in \{0.2, 1\}$. Como *baseline* de comparação, o `GreedyAgent` (sem aprendizado) foi executado por 100 episódios em cada mapa.

2 Resultados

2.1 Iteração de Valor no DiscountGrid

A Tabela 1 apresenta a recompensa acumulada média obtida em 10 episódios para cada um dos cinco MDPs, seguindo a política gulosa derivada de V após 100 iterações.

2.2 CliffGrid: Iteração de Valor vs. Q-Learning

A Tabela 2 compara a recompensa acumulada média sobre 100 episódios para os dois algoritmos, com os parâmetros padrão do simulador.

Tabela 1: Recompensa acumulada média (10 episódios) por configuração de MDP no Discount-Grid.

	γ	Ruído	Living Reward	Retorno médio
MDP 1	0.1	0.0	0	0.0010
MDP 2	0.5	0.2	-2	-3.9854
MDP 3	1.0	0.0	-1	5.0000
MDP 4	0.9	0.2	0	2.9889
MDP 5	1.0	0.2	10	(timeout — episódio não termina)

Tabela 2: Recompensa acumulada média (100 episódios) no CliffGrid.

Algoritmo	Retorno médio
Iteração de Valor	4.2545
Q-Learning	-16.1557

2.3 Q-Learning no Pac-Man

A Tabela 3 resume o desempenho do **PacmanQAgent** após treinamento (política greedy, 10 episódios de teste) e do **GreedyAgent** (100 episódios, sem aprendizado), nos dois mapas.

Tabela 3: Score médio e taxa de vitória por agente/configuração no Pac-Man.

Mapa	Agente	Score médio	Vitórias
smallGrid	Q-Learning ($\alpha = 0.2$)	497.8	10/10
	Q-Learning ($\alpha = 1$)	499.8	10/10
	GreedyAgent	269.1	79/100
tinyCorners	Q-Learning ($\alpha = 0.2$)	-92.2	10/10
	Q-Learning ($\alpha = 1$)	497.0	10/10
	GreedyAgent	-44.1	100/100

3 Análise e Discussão

3.1 DiscountGrid: efeito dos parâmetros na política

Em **MDP 1** ($\gamma = 0.1$): o desconto extremamente baixo faz com que qualquer recompensa a mais de 2-3 passos de distância valha, na prática, quase zero. O agente se torna miope e converge para a saída mais próxima (recompensa 1), mesmo que isso signifique passar perto do penhasco de -10 — já que o ruído é zero, esse risco é inexistente (transições determinísticas). Em **MDP 2** ($\gamma = 0.5$, *living reward* = -2): o alto custo por passo domina o resultado. Mesmo evitando o penhasco, o custo acumulado de viver supera a recompensa da saída mais próxima, resultando em retorno negativo. Isso ilustra como uma penalidade por ação grande o suficiente pode tornar *qualquer* política final custosa, incentivando o agente a terminar o episódio o

quanto antes, independente da saída. Em **MDP 3** ($\gamma = 1$, *living reward* = -1 , ruído = 0): sem desconto, o custo por passo é linear (não exponencial) e o ambiente é determinístico, o que barateia o custo de buscar a saída distante (recompensa 10). O retorno de 5.0 é compatível com atingir essa saída em 5 passos ($10 - 5 \times 1 = 5$), sugerindo que o agente prefere a saída distante mesmo arriscando passar perto do penhasco, já que não há penalização por risco. Em **MDP 4** ($\gamma = 0.9$, ruído = 0.2): com desconto próximo de 1 e ruído presente, o agente ainda tem incentivo a buscar a saída distante, mas o caminho ótimo evita passar perto do penhasco. O retorno intermediário (≈ 2.99) é consistente com atingir a recompensa 10 após um caminho mais longo e mais seguro, com o desconto corroendo parte do valor. Por fim, **MDP 5** ($\gamma = 1$, *living reward* = 10): esta é uma configuração *degenerada* — como não há desconto e cada passo já rende mais recompensa (10) do que qualquer recompensa terminal, não existe incentivo algum para o agente encerrar o episódio. O simulador atingiu o *timeout* do script exatamente pelo motivo esperado: o MDP viola as condições que garantem convergência da Iteração de Valor (é necessário $\gamma < 1$ ou garantia de que todo estado alcança um terminal com probabilidade 1 sob qualquer política).

3.2 CliffGrid: Iteração de Valor vs. Q-Learning

A diferença expressiva entre os retornos (4.25 para VI contra -16.16 para Q-Learning) reflete uma distinção fundamental entre os dois algoritmos: a Iteração de Valor é executada *offline*, isto é, a política gulosa já está totalmente calculada antes do primeiro episódio de simulação começar — logo, todos os 100 episódios simulados usam essa política subótima desde o início. O Q-Learning *aprende enquanto age*: os mesmos 100 episódios usados para calcular a média de retorno são também os episódios de treinamento, nos quais o agente ainda está explorando (via ϵ -greedy) e cometendo erros — inclusive caindo no penhasco repetidamente nas primeiras iterações, quando Q ainda está longe de convergir. A média, portanto, mistura o custo de tempo de aprendizado com o desempenho da política final, o que explica o retorno negativo mesmo que o algoritmo eventualmente aprenda uma política razoável.

3.3 Q-Learning no Pac-Man: efeito de α e do tamanho do domínio

No **smallGrid**, os dois valores de α convergem para políticas de qualidade equivalente após o treinamento (scores de 497.8 e 499.8, ambos com 100% de vitórias), embora a curva de treinamento com $\alpha = 1$ tenha sido visivelmente mais ruidosa (a média das últimas 100 recompensas de treino oscila mais). Isso é esperado: com $\alpha = 1$, cada atualização substitui completamente a estimativa anterior por uma única amostra, sem mediar ruído — o que só não prejudica a convergência final porque o domínio é pequeno o suficiente para que cada par (s, a) seja revisitado muitas vezes ao longo dos 2000 episódios, compensando a alta variância.

No **tinyCorners**, porém, o efeito de α é dramático: com $\alpha = 0.2$ o agente não converge de forma confiável dentro de 2000 episódios. Já com $\alpha = 1$, o agente converge para uma política determinística ótima (todos os 10 episódios de teste com score exato de 497). A explicação mais provável é o tamanho do espaço de estados: **tinyCorners** exige que o agente visite os quatro cantos do mapa, o que multiplica o número de estados distintos (posição $\times$ quais cantos já foram visitados) em relação ao **smallGrid**. Com um espaço de estados maior, cada par (s, a) é visitado menos vezes dentro do mesmo orçamento de episódios, e um α baixo — que

pondera fortemente a estimativa antiga — demora mais atualizações para convergir. Um α alto compensa parcialmente essa escassez de visitas ao dar peso quase total à amostra mais recente, desde que o ambiente não seja excessivamente estocástico (é o caso).

Comparando com o **GreedyAgent**, o Q-Learning bem treinado supera claramente o *baseline* em ambos os mapas, mas o resultado mais revelador é que o Q-Learning *insuficientemente treinado* ($\alpha = 0.2$ no **tinyCorners**, score médio -92.2) teve desempenho *pior* que o próprio **GreedyAgent** (-44.1). Isso reforça que aprendizado por reforço não é estritamente melhor que uma heurística simples *durante* o processo de convergência: a vantagem do Q-Learning só se concretiza depois que a função Q de fato converge para valores próximos dos ótimos, e o tempo necessário para isso depende tanto do tamanho do espaço de estados quanto da taxa de aprendizado escolhida.